

Activités d'Intégration de Compétences — Partie 4

Objectif général

Atteindre un niveau de qualité logicielle professionnel sur le micro service étudiant en mettant en place une stratégie de test complète (unitaire, intégration, E2E, stress), en liant l'outillage de test à la traçabilité Jira via Xray, en intégrant GitHub à Jira pour une visibilité bout-en-bout, et en ajoutant un micro service d'authentification basé sur Express, MongoDB et JWT.

Q1 — Créer une branche Git `version-4` et ajouter le Sprint 4 dans Jira

Indication : Créez la branche `version-4` à partir de `version-3` et ouvrez un nouveau Sprint 4 dans votre projet Jira. Avant de coder, prenez le temps de rédiger les User Stories du sprint : chaque question ci-dessous devrait correspondre à au moins une story. Cette discipline de planification avant exécution est au cœur de la méthodologie Scrum et vous sera utile dans tout contexte professionnel.

```
git checkout version-3
git checkout -b version-4
```

Q2 — Ajouter une stratégie de test complète sur `etudiant-service` (couverture $\geq 80\%$)

Indication : Une bonne stratégie de test repose sur plusieurs niveaux complémentaires, souvent représentés sous forme de pyramide : les tests unitaires constituent la base (rapides, nombreux, isolés), les tests d'intégration le milieu (vérifient la collaboration entre composants), et les tests E2E le sommet (vérifient le comportement complet du système du point de vue de l'utilisateur). Les tests de stress, eux, sont une catégorie transverse qui valide le comportement sous charge. Voici comment aborder chaque niveau.

Tests unitaires

Les tests unitaires vérifient une seule classe en isolation. Utilisez **JUnit 5** et **Mockito** pour simuler les dépendances (repository, mapper). Ciblez en priorité les méthodes de la couche `Service` qui contiennent la logique métier : création d'un étudiant, récupération par année d'inscription, calcul de l'âge, etc. L'annotation `@ExtendWith(MockitoExtension.class)` suffit — pas besoin de charger le contexte Spring complet.

```

@ExtendWith(MockitoExtension.class)
class EtudiantServiceTest {

    @Mock
    private EtudiantRepository repository;

    @Mock
    private EtudiantMapper mapper;

    @InjectMocks
    private EtudiantService service;

    @Test
    void shouldReturnAllEtudiants() {
        // given
        when(repository.findAll()).thenReturn(List.of(new Etudiant()));
        // when
        List<EtudiantDTO> result = service.findAll();
        // then
        assertThat(result).hasSize(1);
    }
}

```

Tests d'intégration avec Testcontainers

Les tests d'intégration vérifient que votre code fonctionne correctement avec une vraie base de données. **Testcontainers** est la solution moderne pour cela : il démarre automatiquement un conteneur PostgreSQL Docker pendant l'exécution des tests, puis le détruit à la fin. Vous n'avez donc pas besoin d'une base de données externe ou d'une base en mémoire (H2) qui ne reflète pas fidèlement le comportement de PostgreSQL. Annotez votre classe de test avec `@SpringBootTest` et `@Testcontainers`.

```

@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
@Testcontainers
class EtudiantIntegrationTest {

    @Container
    static PostgreSQLContainer<?> postgres =
        new PostgreSQLContainer<>("postgres:15")
            .withDatabaseName("testdb")
            .withUsername("test")
            .withPassword("test");

    @DynamicPropertySource
    static void configureProperties(DynamicPropertyRegistry registry) {
        registry.add("spring.datasource.url", postgres::getJdbcUrl);
        registry.add("spring.datasource.username", postgres::getUsername);
        registry.add("spring.datasource.password", postgres::getPassword);
    }

    @Autowired
    private EtudiantRepository repository;

    @Test
    void shouldPersistAndRetrieveEtudiant() {
        Etudiant e = new Etudiant();
        e.setNom("Dupont");
        repository.save(e);
        assertThat(repository.findAll()).isNotEmpty();
    }
}

```

Tests E2E avec Cypress

Les tests End-to-End (E2E) simulent le comportement d'un utilisateur réel dans un navigateur. Avec **Cypress**, vous écrivez des scénarios qui pilotent le frontend Next.js : naviguer vers `/etudiants`, vérifier que la liste s'affiche, remplir le formulaire de création, soumettre, et vérifier que le nouvel étudiant apparaît. Ces tests supposent que toute la stack (frontend, API Gateway, micro services) est lancée, via `docker compose up`. Placez vos tests Cypress dans un dossier `cypress/e2e/` à la racine du projet `frontend/`.

```
// cypress/e2e/etudiants.cy.js
describe('Gestion des étudiants', () => {
  it('affiche la liste des étudiants', () => {
    cy.visit('http://localhost:3000/etudiants');
    cy.get('[data-testid="etudiant-list"]').should('be.visible');
    cy.get('[data-testid="etudiant-item"]').should('have.length.greaterThan', 0);
  });

  it('crée un nouvel étudiant', () => {
    cy.visit('http://localhost:3000/etudiants/nouveau');
    cy.get('[name="nom"]').type('Alice Martin');
    cy.get('[name="cin"]').type('12345678');
    cy.get('[type="submit"]').click();
    cy.contains('Alice Martin').should('be.visible');
  });
});
```

Tests de stress avec Gatling

Les tests de stress évaluent le comportement de votre API sous charge. **Gatling** est l'outil standard dans l'écosystème Java pour cela. Créez un scénario qui simule plusieurs utilisateurs concurrents appelant `GET /api/etudiants` pendant une durée définie. L'objectif est de mesurer le temps de réponse moyen, le percentile 95, et d'identifier les goulots d'étranglement. Ajoutez Gatling comme plugin Maven dans votre `pom.xml`.

```
// src/gatling/java/simulations/EtudiantSimulation.java
public class EtudiantSimulation extends Simulation {

  HttpProtocolBuilder httpProtocol = http
    .baseUrl("http://localhost:8080")
    .acceptHeader("application/json");

  ScenarioBuilder scn = scenario("Liste des étudiants")
    .exec(http("GET /api/etudiants")
      .get("/api/etudiants")
      .check(status().is(200)));

  { // bloc d'initialisation
    setUp(scn.injectOpen(
      rampUsers(50).during(Duration.ofSeconds(30))
    ).protocols(httpProtocol);
  }
}
```

Mesure de la couverture

Configurez le plugin **JaCoCo** dans votre `pom.xml` pour mesurer la couverture de code. Ajoutez un seuil minimum de 80 % : si la couverture descend en dessous, le build Maven échoue automatiquement, ce qui force l'équipe à maintenir la qualité.

```
<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <version>0.8.11</version>
  <executions>
    <execution>
      <goals><goal>prepare-agent</goal></goals>
    </execution>
    <execution>
      <id>check</id>
      <goals><goal>check</goal></goals>
      <configuration>
        <rules>
          <rule>
            <limits>
              <limit>
                <counter>LINE</counter>
                <value>COVEREDRATIO</value>
                <minimum>0.80</minimum>
              </limit>
            </limits>
          </rule>
        </rules>
      </configuration>
    </execution>
  </executions>
</plugin>
```

Q3 — Utiliser Testcontainers pour PostgreSQL

Indication : Cette question est traitée conjointement avec Q2 (tests d'intégration). Pour être complet, assurez-vous d'ajouter les dépendances Maven nécessaires dans votre `pom.xml`. Testcontainers nécessite que Docker soit actif sur la machine qui exécute les tests. En CI/CD (GitHub Actions par exemple), Docker est disponible par défaut sur les runners Ubuntu, ce qui rend cette approche parfaitement compatible avec l'intégration continue.

```
<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>junit-jupiter</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>postgresql</artifactId>
  <scope>test</scope>
</dependency>
```

Q4 — Ajouter l'intégration GitHub ↔ Jira

Indication : L'intégration GitHub ↔ Jira permet de lier automatiquement vos commits, branches et Pull Requests aux tickets Jira correspondants, offrant ainsi une traçabilité complète entre le code et le besoin métier. Voici comment la mettre en place et l'utiliser correctement.

Dans Jira, installez l'application **GitHub for Jira** depuis le Jira Marketplace (disponible gratuitement sur Jira Cloud). Autorisez l'accès à votre dépôt GitHub. Une fois l'intégration active, Jira détecte automatiquement les références à des clés de tickets dans vos commits et PR.

La convention à respecter est simple : incluez la clé du ticket Jira (ex. PROJ-23) dans le message de chaque commit, dans le nom de chaque branche, et dans le titre de chaque Pull Request. Jira affichera alors automatiquement les commits, branches et PRs associés directement dans la fiche du ticket.

```
# Convention de nommage de branche
git checkout -b feature/PROJ-23-ajout-tests-unitaires

# Convention de message de commit
git commit -m "PROJ-23 : ajout des tests unitaires EtudiantService avec Mockito"

# Le titre de la PR reprend également la clé
# "PROJ-23 : Tests unitaires et couverture JaCoCo >= 80%"
```

Q5 — Ajouter l'intégration Xray pour la gestion des cas de test

Indication : **Xray** est un plugin Jira spécialisé dans la gestion des tests. Il permet de créer des cas de test directement dans Jira, de les associer à des User Stories, d'exécuter des campagnes de test, et de publier automatiquement les résultats depuis votre pipeline CI. L'objectif est d'avoir une **couverture fonctionnelle** visible dans Jira : pour chaque User Story, on sait quels tests la couvrent et si ces tests passent.

Voici le workflow à mettre en place. Installez d'abord **Xray for Jira** (version Cloud gratuite disponible). Créez ensuite des **Test** (type d'issue Xray) pour chacune de vos méthodes de test JUnit importantes. Liez ces Tests à leurs User Stories via le champ "Test Coverage". Créez un **Test Plan** regroupant tous les tests du Sprint 4, puis une **Test Execution** pour chaque run de votre pipeline CI.

Pour publier automatiquement les résultats JUnit dans Xray depuis GitHub Actions, utilisez le rapport XML généré par Maven Surefire et l'API REST Xray.

```
# .github/workflows/test-and-report.yml (extrait)
- name: Run tests
  run: mvn verify

- name: Publish results to Xray
  run: |
    curl -H "Content-Type: multipart/form-data" \
      -H "Authorization: Bearer ${ secrets.XRAY_TOKEN }" \
      -F "file=@target/surefire-reports/TEST-*.xml" \
        "https://xray.cloud.getxray.app/api/v2/import/execution/junit?
projectKey=PROJ"
```

Q6 — Ajouter un micro service Authentification avec Express, MongoDB et JWT

Indication : Ce micro service introduit une technologie différente (Node.js / Express) dans votre architecture, ce qui est tout à fait réaliste dans un contexte de micro services : chaque service peut être développé avec la technologie la plus adaptée à son rôle. Ce service gère l'inscription et la connexion des utilisateurs, et émet des tokens JWT que les autres micro services pourront valider.

Créez un projet Node.js dans un dossier `auth-service/`. Utilisez **Express** pour l'API REST, **Mongoose** pour la modélisation MongoDB, **bcrypt** pour le hachage des mots de passe, et **jsonwebtoken** pour l'émission et la vérification des tokens. Exposez deux endpoints principaux : `POST /auth/register` et `POST /auth/login`. Ce service doit être ajouté au `docker-compose.yml` avec un conteneur MongoDB dédié.

```
// auth-service/src/routes/auth.js
const express = require('express');
const bcrypt = require('bcrypt');
const jwt = require('jsonwebtoken');
const User = require('../models/User');

const router = express.Router();

// Inscription
router.post('/register', async (req, res) => {
  const { username, password } = req.body;
  const hashed = await bcrypt.hash(password, 10);
  const user = await User.create({ username, password: hashed });
  res.status(201).json({ id: user._id, username: user.username });
});

// Connexion
router.post('/login', async (req, res) => {
  const { username, password } = req.body;
  const user = await User.findOne({ username });
  if (!user || !(await bcrypt.compare(password, user.password))) {
    return res.status(401).json({ message: 'Identifiants invalides' });
  }
  const token = jwt.sign(
    { userId: user._id, username: user.username },
    process.env.JWT_SECRET,
    { expiresIn: '1h' }
  );
  res.json({ token });
});

module.exports = router;
```

Le modèle `User` avec Mongoose est le suivant :

```
// auth-service/src/models/User.js
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  username: { type: String, required: true, unique: true },
  password: { type: String, required: true }
}, { timestamps: true });

module.exports = mongoose.model('User', userSchema);
```

Mettez à jour le `docker-compose.yml` pour ajouter les deux nouveaux services :


```

mongodb:
  image: mongo:7
  ports:
    - "27017:27017"

auth-service:
  build: ./auth-service
  ports:
    - "3001:3001"
  depends_on:
    - mongodb
  environment:
    MONGO_URI: mongodb://mongodb:27017/authdb
    JWT_SECRET: votre_secret_jwt_ici

```

Structure du dépôt GitHub attendue

```

/projet-etudiants/
├── api-spring-boot/
│   ├── src/
│   │   ├── main/
│   │   └── test/
│   │       ├── java/
│   │       │   ├── unit/           # Tests unitaires JUnit + Mockito
│   │       │   └── integration/    # Tests d'intégration Testcontainers
│   │       └── resources/
│   │           └── features/       # Fichiers Gherkin (Partie 2)
│   └── pom.xml                    # Avec JaCoCo + Testcontainers + Gatling
├── auth-service/                  # Micro service Node.js (nouveau)
│   ├── src/
│   │   ├── models/User.js
│   │   ├── routes/auth.js
│   │   └── app.js
│   └── package.json
├── frontend/
│   └── cypress/
│       └── e2e/                   # Tests E2E Cypress (nouveau)
├── .github/
│   ├── workflows/
│   │   └── test-and-report.yml    # Pipeline CI avec publication Xray
│   ├── ISSUE_TEMPLATE/
│   └── pull_request_template.md
└── docker-compose.yml            # Mis à jour avec mongodb + auth-service

```

💡 **Rappel** : la couverture de 80 % est un seuil minimum, pas un objectif en soi. Un test mal écrit qui couvre du code sans rien vérifier est pire qu'un test absent, car il donne une fausse confiance. Concentrez-vous sur la **pertinence** des assertions autant que sur le taux de couverture. C'est cette combinaison — couverture quantitative et qualité des assertions — qui définit une véritable culture de la qualité logicielle.